Pricing Baku Real Estate from Scratch

Decision trees and Pegasos SVMs on bina.az listings

Melek Askerova Sevda Gurbanova Nezrin Ibrahimova

MLE-AI-201 Machine Learning, AI Academy

`github.com/nezrinibrahimova/ML_Final_Project` (tag `v1.0-final`)

Problem and dataset

Two tasks on one dataset

- **Task A**: predict the asking price (regression)
- **Task B**: premium vs. standard premium $=$ price $>$ *train* median (208,000 AZN)

Rules

- Tree and SVM written **from scratch** in NumPy
- scikit-learn only as a baseline

bina.az sale listings (Kaggle)

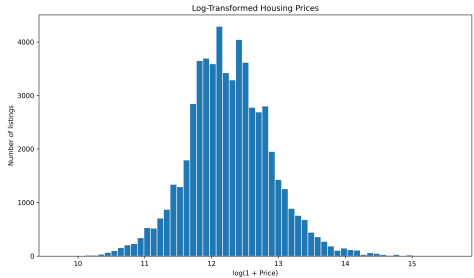
- 100,775 rows $\times$ 51 columns
- scraped 5 Oct – 19 Nov 2024, all in AZN
- price 16.5k – 5.95M AZN

58,007 listings after cleaning
148 features **50/50** classes

Cleaning, features and leakage

A

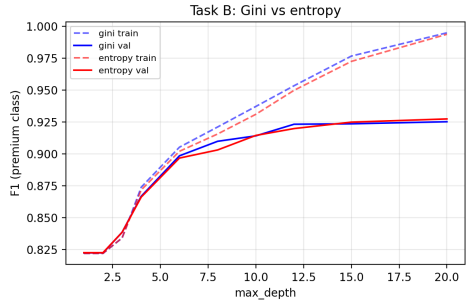
- Residential only (new, old, house): 91,567
- Latest record per listing URL: 58,452
- Outliers: area 15–1000 m^2, price/m^2 300–10k, valid coordinates: **58,007**
- Features: area, rooms, floors, land, lat/lng, repair/deed/mortgage + one-hot category and district
- Leakage removed: `unit_price`, `total_price`; no owner/shop names or address
- Price skew 5.79 $\rightarrow$ 0.27 after $\log \Rightarrow$ tree learns $\log(\text{price})$



Our decision tree

B

- Criteria: **Gini**, **entropy** (Task B), **MSE** (Task A)
- Split = max impurity decrease $\Delta I = I(P) - \frac{N_L}{N} I(L) - \frac{N_R}{N} I(R)$
- **Vectorised search**: sort once, cumulative counts / sums $\Rightarrow$ all thresholds scored at once
- 4 controls: `max_depth`, `min_samples_split`, `min_samples_leaf`, `min_impurity_decrease`
- `predict_proba`, `feature_importances_`



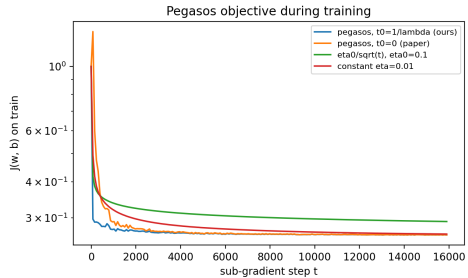
Train (dashed) vs. validation F1: deeper $\Rightarrow$ overfits, validation flattens after depth 12.

Our Pegasos SVM

C

$$J(\mathbf{w}, b) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_i \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b))$$

- Mini-batch sub-gradient; hinge-active points push $\mathbf{w}$
- Fix: paper's $\eta_t = 1/(\lambda t)$ throws the bias to $\pm$hundreds $\Rightarrow \eta_t = 1/(\lambda(t + 1/\lambda))$
- Reports $\|\mathbf{w}\|$, margin $2/\|\mathbf{w}\|$, support vectors
- **Non-linear**: random Fourier features $\phi(\mathbf{x}) \cdot \phi(\mathbf{x}') \approx e^{-\gamma\|\mathbf{x}-\mathbf{x}'\|^2}$, cost $O(nD)$ not $O(n^2)$



Objective decreases; the paper schedule (orange) spikes first.

Experimental setup

One protocol for every model

- Same stratified split 70/15/15 (seed 42)
  40,605 / 8,701 / 8,701
- Threshold and scaling from **train only**
- Hyperparameters chosen on **validation**
- Test scored **once**

Fair baselines

- sklearn trees: same depth / leaf / criterion
- `LinearSVC`, SGD: $C = 1/(\lambda n) \Rightarrow$ same objective
- RBF `SVC` (10k subsample)
- 5-fold CV mean $\pm$ std, fit/predict time

Metrics (own NumPy code): RMSE, MAE, R^2 | precision, recall, F1, ROC-AUC
Reproducible: `python -m src.run_all` rebuilds everything in $\sim$24 min; 49 unit tests

Results (test set)

Task A: price

Model	RMSE	R^2
Mean baseline	222,308	0.000
Ridge (bonus)	124,377	0.687
Our tree	**119,931**	**0.709**
sklearn tree	119,699	0.710

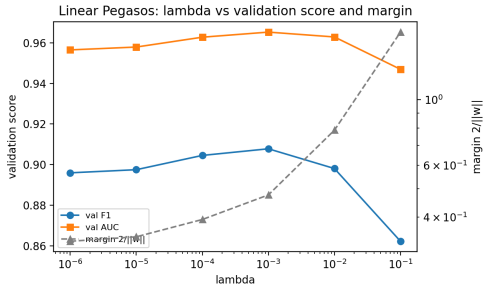
Our models $\approx$ sklearn
(tree R^2 0.709 vs. 0.710;
SVM: same objective value)

Task B: price tier

Model	F1	AUC	CV F1
Our tree (entropy)	**.926**	.930	.920$\pm$.003
Our tree (Gini)	.923	.954	.920$\pm$.004
sklearn tree (entropy)	.925	.929	.920$\pm$.004
Our Pegasos + RFF	.912	**.974**	–
Our Pegasos linear	.905	.964	.904$\pm$.005
sklearn `LinearSVC`	.906	.964	.905$\pm$.005
sklearn RBF `SVC`	.912	.973	–

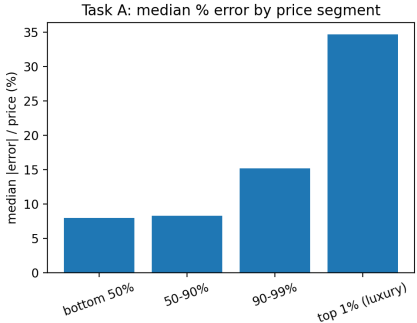
What we learned

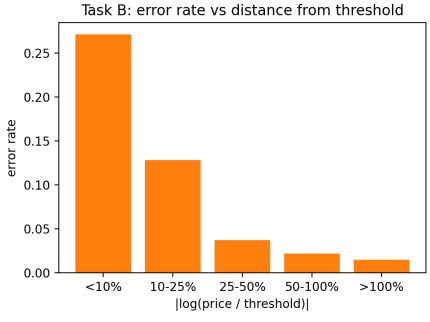
- **Tree wins F1**: price tier = area $\times$ district interaction, axis splits capture it
- **SVM wins AUC**: continuous margin ranks listings smoothly
- **Margin trade-off**: larger $\lambda \Rightarrow$ wider margin, more support vectors; best at $\lambda = 10^{-3}$
- Scaling matters: standardised rare-district dummies ($|x| \approx 200$) break RBF; RFF val F1 **.859** $\rightarrow$ **.913** with 0/1 dummies



Linear Pegasos: validation score and margin vs. λ.

Where the models fail





- **Luxury**: median error 8% $\rightarrow$ 35% for the top 1%; under-priced by $\sim$38%
- **Tier border**: 27% errors within 10% of 208k, $<$4% far away
- **Rare districts**: 56 of 138 have $<$50 listings (Xaçmaz: 122% error)
- **Limits**: asking $\neq$ sale price; 6 weeks of data; our tree 7$\times$ slower than sklearn

Who did what, and next steps

A — Nezrin Ibrahimova
data cleaning, leakage, split, EDA, metrics;
bonus: Ridge, PCA + k-means segments

B — Sevda Gurbanova
decision tree, sweeps, learning curves,
sklearn comparison, CV, error analysis

C — Melek Askerova
Pegasos SVM + RFF, SVM baselines,
`run_all`, packaging, reproducibility

Next

- Random forest of our own trees (cut variance)
- Train-only district statistics for rare districts
- Separate model for luxury listings

Thank you!

Questions?